Agent Is All You Need: Condition Number Barriers in Deep Logic Learning and the Optimization Advantage of Agentic Architectures

Daniel Derycke (d.deryckeh@gmail.com)
Date: 5 May 2026

Acknowledgments: Substantial writing assistance, technical review, and annotation were provided by Claude Opus 4.6, and Gemini 3.1 under the sole direction and oversight of the author.

Abstract. We investigate the optimization-theoretic barriers that gradient-based methods face when learning exact combinatorial logic of depth D. We first demonstrate that smooth, low-Lipschitz representations of deep Boolean functions *exist* (the multilinear extension has $L = O(\sqrt{N})$), establishing that the barrier is not representational but *optimizational*. We then prove the *Condition Number Sub-Multiplicativity Theorem*: end-to-end backpropagation through a depth-D composition yields an aggregate condition number bounded by $\kappa_{\text{total}} \leq \prod_{i=1}^{D} \kappa_i$, and show via the Furstenberg–Kesten theorem that this bound is generically achieved, giving $\kappa = e^{\Theta(D)}$ with high probability under standard initialization. For recurrent architectures, we prove the *Recurrent Dilemma*: maintaining discrete logical states requires either spectral radius $\lambda_{\max} > 1$ (exponential gradient explosion via BPTT) or $\lambda_{\max} = 1$ with polynomial center-manifold escape (severe latency and polynomial gradient explosion). We show these barriers persist under SGD, momentum, and Adam. Finally, we prove the *Optimization Factorization Theorem*: intermediate supervision — whether via discrete tools or continuous checkpoints — decomposes a single depth-D problem with $\kappa \propto c^D$ into K independent sub-problems with $\kappa \propto c^{D/K}$ each, reducing total optimization cost from exponential to polynomial. This provides an optimization-theoretic justification for the empirical superiority of agentic AI architectures.

Keywords: condition number, optimization factorization, depth separation, agentic AI, intermediate supervision

MSC 2020: 68T07, 90C26, 37C75, 68Q17

Summary of Main Results

#	Result	Statement	Reference
1	Representation is not the barrier	Multilinear extension gives $L \leq \sqrt{N}$ for any Boolean f	Theorem 3.1
2	Condition number sub-multiplicativity	$\kappa(\prod J_i) \leq \prod \kappa(J_i)$	Theorem 4.1
3	Generic exponential conditioning	$\kappa = e^{\Theta(D)}$ w.h.p. under standard init.	Theorem 4.3, Corollary 4.4
4	Recurrent dilemma	Explosion ($\lambda > 1$) or vanishing ($\lambda = 1$, center manifold)	Theorem 5.1
5	First-order persistence	SGD, momentum, Adam all $\Omega(e^{\Theta(D/2)})$	Theorems 6.1–6.3, Corollary 6.4
6	Optimization factorization	K checkpoints: cost $K \cdot c^{D/K}$ vs. c^D	Theorem 7.1
7	Necessity of $\kappa_i > 1$	Isometric layers ($\kappa = 1$) cannot compute nontrivial logic	Theorem 9.1
8	Approximate learnability boundary	AC^0: quasi-poly learnable; outside AC^0: exponential	Theorem 9.3
9	Second-order methods	Newton removes κ from iterations but not from total cost	Theorem 9.8

1. Introduction

The scaling hypothesis posits that increasing neural network capacity will eventually yield arbitrary reasoning. The Universal Approximation Theorem [5, 9] guarantees that suitable weights *exist*. But can gradient descent *find* them?

This paper addresses this question. We establish three results:

1. **The barrier is not representational** (§3): Deep Boolean functions admit smooth, polynomial-Lipschitz continuous extensions. The UAT is not the bottleneck.
2. **The barrier is optimizational** (§4–5): End-to-end gradient-based training of depth-D logic suffers exponential condition number growth $\kappa \propto c^D$, making optimization intractable — not impossible, but requiring exponential time.

3. **Intermediate supervision breaks the barrier** (§7): Providing ground-truth intermediate results — via discrete tools, chain-of-thought, or continuous targets — factorizes the condition number from c^D to $D \cdot c^{D/K}$, reducing optimization from exponential to polynomial.

1.1 What We Do NOT Claim

We explicitly disclaim: - ~~Neural networks cannot *represent* deep logic~~ — they can (Theorem 3.1). - ~~Hardware underflow is the fundamental barrier~~ — while finite-precision limits are quickly exceeded, the condition number barrier is theoretically fundamental and persists even with infinite-precision arithmetic. - ~~Agentic tool-use is the *unique* solution~~ — continuous intermediate supervision also works (Remark 7.2).

What we *do* claim: end-to-end training is *exponentially slower* than factorized training, and agentic architectures are the most natural instantiation of factorized training in practice.

1.2 Relation to Prior Work

Hochreiter [8], Bengio et al. [2], and Pascanu et al. [15] identified gradient instability in deep/recurrent networks. Balduzzi et al. [1] showed gradients in deep networks resemble white noise. Our contribution: we prove that *condition number multiplication* is the unifying mechanism, and that *intermediate supervision* is the principled remedy. Telgarsky [17] and Eldan–Shamir [6] proved depth separation. We build on these to show that the optimization cost of depth is exponential in D.

---

2. Preliminaries

Definition 2.1 (Feedforward Network). $\mathcal{N}_\theta(\mathbf{x}) = (F_D \circ \cdots \circ F_1)(\mathbf{x})$, $F_i(\mathbf{z}) = \sigma(W_i\mathbf{z} + \mathbf{b}_i)$, parameters $\theta \in \mathbb{R}^P$.

Definition 2.2 (Recurrent Network). Shared-weight iteration $\mathbf{h}_{t+1} = \Phi_\theta(\mathbf{h}_t)$, output $y = \psi(\mathbf{h}_T)$.

Definition 2.3 (Robust Boolean Emulation). $|\mathcal{N}_\theta(\mathbf{x}) - f(\mathbf{v})| < 1/3$ for $\mathbf{x} \in B_\delta(\mathbf{v})$, all $\mathbf{v} \in \{0,1\}^N$.

Definition 2.4 (Condition Number). For an L-smooth, μ-strongly convex loss: $\kappa = L/\mu$. Gradient descent converges in $O(\kappa \log(1/\varepsilon))$ iterations [13].

Theorem 2.5 (Telgarsky [17]). *Depth-D ReLU networks compute functions requiring width $2^{\Omega(D)}$ at lower depth.*

Theorem 2.6 (Cybenko [5], Hornik et al. [9]). *Single-hidden-layer networks can approximate any continuous function.*

3. The Representation Is Not the Barrier

We first establish that smooth, low-Lipschitz continuous extensions of deep Boolean functions *exist*. This rules out any "unconditional impossibility" argument.

Theorem 3.1 (Polynomial Lipschitz Representations Exist). *For any Boolean function $f : \{0,1\}^N \to \{0,1\}$, regardless of logical depth, there exists a continuous extension $\hat{f} : [0,1]^N \to [0,1]$ that robustly emulates f with global Lipschitz constant $L(\hat{f}) \leq \sqrt{N}$.*

Proof. The multilinear extension of f is:

$$\hat{f}(\mathbf{x}) = \sum_{\mathbf{v} \in \{0,1\}^N} f(\mathbf{v}) \prod_{i=1}^{N} x_i^{v_i} (1 - x_i)^{1 - v_i}$$

This is a multilinear polynomial that exactly interpolates f on the Boolean hypercube: $\hat{f}(\mathbf{v}) = f(\mathbf{v})$ for all $\mathbf{v} \in \{0,1\}^N$. Each partial derivative is:

$$\frac{\partial \hat{f}}{\partial x_j}(\mathbf{x}) = \sum_{\mathbf{v}} f(\mathbf{v}) \cdot \frac{v_j - (1 - v_j)}{1} \cdot \prod_{i \neq j} x_i^{v_i} (1 - x_i)^{1 - v_i}$$

A multilinear polynomial is strictly of degree 1 (linear) with respect to any single variable x_j. Therefore, its partial derivative $\frac{\partial \hat{f}}{\partial x_j}$ is simply the difference $\hat{f}(x_j = 1, \mathbf{x}_{-j}) - \hat{f}(x_j = 0, \mathbf{x}_{-j})$. Because the multilinear extension maps the Boolean hypercube strictly into its convex hull $[0,1]$, both of these evaluations lie in $[0,1]$. Their maximum difference is strictly bounded by $1 - 0 = 1$. Thus, $|\frac{\partial \hat{f}}{\partial x_j}| \leq 1$ globally for all j. With N variables, the total gradient norm is bounded by $\|\nabla \hat{f}\| \leq \sqrt{N \cdot 1^2} = \sqrt{N}$. $\square$

Remark 3.2. Theorem 3.1 rules out any claim that deep logic *requires* exponential Lipschitz constants. The barrier is not in the existence of smooth solutions, but in whether gradient descent can *find* them among the exponentially many local minima and saddle points of the parameter landscape.

Remark 3.3 (Inductive Bias of Standard Architectures). While polynomial-Lipschitz solutions exist in function space, standard ReLU/GeLU networks with random initialization parameterize a *different* region of function space. With standard initialization, the right spectral edge dictates $\sigma_{\max}(W_i) \to 2$ (by the Marchenko-Pastur law, Theorem 4.3). Thus, the product operator norm grows rapidly as $\| \prod_i W_i \| \leq 2^D$. However, the true barrier is not merely operator norm growth, but the exponential separation between the largest and smallest singular values—the Lyapunov exponent spread. The smooth multilinear solutions occupy a measure-zero subset of the parameter space, and gradient descent has no inductive bias toward finding them.

4. The Condition Number Barrier

4.1 Jacobian Product and Conditioning

Theorem 4.1 (Condition Number Sub-Multiplicativity). *Let $f = g_D \circ \cdots \circ g_1$ be a composition of D differentiable maps where all hidden layers have uniform width w. Consequently, the per-layer Jacobians $J_i = Dg_i$ are square endomorphisms. The condition number of the end-to-end Jacobian satisfies:*

$$\kappa\left(\prod_{i=1}^{D} J_i\right) \leq \prod_{i=1}^{D} \kappa(J_i)$$

Proof. Standard sub-multiplicativity of condition numbers: $\sigma_{\max}(AB) \leq \sigma_{\max}(A)\sigma_{\max}(B)$ and $\sigma_{\min}(AB) \geq \sigma_{\min}(A)\sigma_{\min}(B)$ (the latter when all matrices are invertible). Therefore $\kappa(AB) = \sigma_{\max}/\sigma_{\min} \leq (\sigma_{\max}(A)\sigma_{\max}(B))/(\sigma_{\min}(A)\sigma_{\min}(B)) = \kappa(A)\kappa(B)$. □

Remark 4.2 (Direction of the Bound). The sub-multiplicativity gives an *upper* bound: $\kappa_{\text{end}} \leq \prod \kappa_i$. In special cases, cancellation can occur: if A stretches direction $\hat{e}_1$ and B stretches $\hat{e}_2$, then AB may be nearly isometric ($\kappa(AB) \approx 1$). The critical question is whether such cancellation occurs *generically* in trained neural networks.

Theorem 4.3 (Generic Condition Number Growth). *Let $W_1, \ldots, W_D \in \mathbb{R}^{w \times w}$ be independent random matrices with entries drawn from standard initialization (e.g., $\mathcal{N}(0, 1/w)$). By Edelman (1988) [12], each layer strictly isolates $\kappa(W_i) = \Theta(w)$. For the product $\prod_i W_i$:*

(i) The logarithm of the condition number concentrates: $\log \kappa(\prod_i W_i) = \Theta(D)$ almost surely over the initialization.

(ii) In the infinite-width mean-field limit, the Lyapunov exponents of the full non-linear network Jacobian $J_{\mathbf{x}} = \prod D_i W_i$ satisfy $\lambda_1 - \lambda_w = \Theta(1)$ per step [24, 25], meaning the gap accumulates as $\Theta(D)$, giving $\kappa = e^{\Theta(D)}$.

Proof. We invoke three classical results from the theory of products of random matrices.

Step 1 (Furstenberg–Kesten [18]). Let $W_1, W_2, \ldots$ be i.i.d. random matrices in $\mathbb{R}^{w \times w}$ with $\mathbb{E}[\log^+ \|W_1\|] < \infty$. Then the top Lyapunov exponent

$$\lambda_1 := \lim_{D \to \infty} \frac{1}{D} \log \|W_D \cdots W_1\|$$

exists almost surely and equals $\inf_D \frac{1}{D}\mathbb{E}[\log \|W_D \cdots W_1\|]$. For W_i with entries $\sim \mathcal{N}(0, 1/w)$, the integrability condition is satisfied since $\mathbb{E}[\log \|W_i\|] \leq \log \mathbb{E}[\|W_i\|] < \infty$ by Jensen's inequality and the finiteness of the expected operator norm.

Step 2 (Oseledets Multiplicative Ergodic Theorem [19]). Under the same conditions, the limit

$$\Lambda := \lim_{D\to\infty} \left[(W_D \cdots W_1)^\top (W_D \cdots W_1)\right]^{1/(2D)}$$

exists almost surely. Its eigenvalues $e^{\lambda_1} \geq e^{\lambda_2} \geq \cdots \geq e^{\lambda_w}$ define the full Lyapunov spectrum. Equivalently, the singular values of the product satisfy:

$$\lim_{D\to\infty} \frac{1}{D} \log \sigma_k \left(\prod_{i=1}^{D} W_i \right) = \lambda_k \quad \text{a.s.}, \quad k = 1, \ldots, w$$

Therefore $\lim_{D\to\infty} \frac{1}{D} \log \kappa \left(\prod_i W_i\right) = \lambda_1 - \lambda_w$ almost surely.

Step 3 (Furstenberg's Positivity Theorem [20]). The Lyapunov exponent gap $\lambda_1 - \lambda_w > 0$ if and only if the smallest closed subgroup G_μ generated by the support of the distribution of W_i is (a) non-compact and (b) strongly irreducible (no finite union of proper subspaces is G_μ-invariant). For W_i with i.i.d. Gaussian entries, $G_\mu = \mathrm{GL}(w, \mathbb{R})$, which is non-compact and acts irreducibly on $\mathbb{R}^w$. Therefore $\lambda_1 - \lambda_w =: \gamma > 0$.

Conclusion. Combining Steps 1–3:

$$\log \kappa \left(\prod_{i=1}^{D} W_i \right) = \gamma D + o(D) \quad \text{a.s.}$$

where $\gamma = \lambda_1 - \lambda_w > 0$ is a constant depending only on the distribution of W_i (and hence on w and the initialization scheme). By concentration inequalities for products of random matrices [21], the $o(D)$ term is $O(\sqrt{D \log D})$ with high probability, giving $\kappa = e^{\Theta(D)}$ for the linear product. To bridge this to the full network Jacobian $J_{\mathbf{x}} = \prod D_i W_i$, we invoke the infinite-width mean-field limit [24, 25]. In wide networks, the pre-activations act as stationary Gaussian noise, rendering the activation matrices D_i approximately i.i.d. The mean-field theory rigorously establishes that the Lyapunov spectrum gap $\gamma > 0$ generically persists asymptotically in the full non-linear product, preserving the exponential condition number growth. □

Corollary 4.4 (Parameter Landscape Ill-Conditioning via the NTK). *The exponential ill-conditioning of the input-output Jacobian $J_{\mathbf{x}}$ directly dictates the conditioning of the optimization landscape over a dataset. The non-null subspace of the parameter Gauss-Newton Hessian $H \approx J_\theta^\top J_\theta$ shares the exact non-zero eigenvalue spectrum of the empirical Neural Tangent Kernel $K = J_\theta J_\theta^\top = \sum_{k=1}^{D} J_{\theta_k} J_{\theta_k}^\top$. Because adding positive semi-definite matrices strictly increases the maximum eigenvalue, the top eigenvalue of the NTK is lower-bounded by the first layer's contribution: $\lambda_{\max}(K) \geq \lambda_{\max}(J_{\theta_1} J_{\theta_1}^\top) = e^{\Theta(D)}$ (driven by the top Lyapunov exponent). However, this exponential stretching is strictly confined to the dominant Lyapunov vector (the unstable manifold). For a dataset of size $N \geq 2$, the remaining orthogonal directions in the NTK do not experience this*

explosion. Instead, the minimum eigenvalue of the NTK is anchored by the well-conditioned shallow layers near the output: $\lambda_{\min}(K) \geq \lambda_{\min}(J_{\theta_D} J_{\theta_D}^\top) = \mathcal{O}(1)$. *Therefore, the condition number of the non-null optimization landscape inherits this exponential divergence:* $\kappa(H_{\text{non-null}}) = \kappa(K) \geq e^{\Theta(D)}/\mathcal{O}(1) = e^{\Theta(D)}$. *Gradient descent strictly requires* $\Omega(e^{\Theta(D)})$ *iterations.*

Remark 4.5 (Honest Assessment). We emphasize: - The exponential conditioning is a *generic* (probabilistic) property, not an unconditional lower bound. - Specially designed initializations (dynamical isometry, orthogonal initialization) can achieve $\kappa_i \approx 1$, forestalling the exponential growth. However, maintaining $\kappa_i = 1$ throughout training is not guaranteed once optimization begins modifying the weights. - The key insight is that *standard training practice* (random initialization + first-order optimization) generically produces exponential conditioning, which intermediate supervision can circumvent.

---

5. The Recurrent Dilemma

For shared-weight architectures (RNNs, looped Transformers), the state map $\Phi_\theta : \mathbb{R}^W \to \mathbb{R}^W$ is a square endomorphism, so eigenvalues and spectral radius are well-defined.

Theorem 5.1 (The Shattering-or-Latency Dilemma). *Let* $\Phi_\theta : \mathbb{R}^W \to \mathbb{R}^W$ *maintain two attracting basins encoding logical 0 and 1, separated by a boundary* $\mathcal{M}_U$. *At any fixed point* $\mathbf{h}^* \in \mathcal{M}_U$, *let* $\lambda_{\max} = \rho(D\Phi_\theta(\mathbf{h}^*))$. *Then:*

(i) If $\lambda_{\max} > 1$ *(hyperbolic case): The BPTT maximum singular value* $\sigma_{\max}(\prod J_t)$ *explodes as* $\Omega(\lambda_{\max}^T)$. *Since attractor basins strictly require contracting stable manifolds, the minimum singular value vanishes as* $\sigma_{\min}(\prod J_t) \leq \mathcal{O}(\lambda_{\min}^T)$. *The condition number therefore strictly satisfies* $\kappa \geq \Omega((\lambda_{\max}/\lambda_{\min})^T)$, *which is exponential in* T.

(ii) If $\lambda_{\max} = 1$ *(center manifold case): The trajectory escapes* $\mathcal{M}_U$ *via nonlinear terms in time* $T = \mathcal{O}(1/\varepsilon^{p-1})$ *where* p *is the order of the leading nonlinear term. The gradient through this escape explodes polynomially as* $\mathcal{O}(T^{\frac{p}{p-1}})$.

(iii) Case (i) results in immediate exponential explosion of gradients in T. *Case (ii) results in severe sequence latency coupled with polynomial explosion of gradients. In either case, maintaining long discrete logic chains via shared weights is analytically intractable.*

Proof.

(i) With $\lambda_{\max} > 1$, we invoke the Stable Manifold Theorem (Hadamard–Perron). In the unstable subspace of the fixed point $D\Phi_\theta(\mathbf{h}^*)$, the linearized map yields exponential growth $\|D\Phi_\theta(\mathbf{h}^*)^T v\| \geq \lambda_{\max}^T \|v\|$. By local topological conjugacy (Hartman–Grobman) or cone-invariance, this exponential expansion persists for the nonlinear trajectory product $\prod_{t=0}^{T-1} J_t$ along the unstable manifold,

guaranteeing $\sigma_{\max}(\prod_{t=0}^{T-1} J_t) = \Omega(\lambda_{\max}^T)$. Concurrently, the discrete dynamical system requires an attracting stable manifold to robustly encode logic states, meaning the minimum singular value is bounded by the contraction rate $\sigma_{\min}(\prod_{t=0}^{T-1} J_t) \leq \mathcal{O}(\lambda_{\min}^T)$ with $\lambda_{\min} < 1$. The ratio yields an exponentially exploding condition number $\kappa \geq \Omega((\lambda_{\max}/\lambda_{\min})^T)$.

(ii) When $\lambda_{\max} = 1$, the fixed point is non-hyperbolic. By the Discrete Center Manifold Theorem [10], the dynamics near $\mathbf{h}^*$ reduce to the normal form $h_{t+1} = h_t + ah_t^p + \mathcal{O}(h_t^{p+1})$ for some leading nonlinearity $p \geq 2$. Escaping from an initial perturbation $h_0 = \varepsilon$ to an $\mathcal{O}(1)$ state requires discrete time $T = \mathcal{O}(\varepsilon^{1-p})$. The end-to-end gradient over this discrete trajectory is the product $\frac{\partial h_T}{\partial h_0} = \prod_{t=0}^{T-1}(1 + aph_t^{p-1})$. Utilizing the discrete trajectory approximation $h_t \sim (h_0^{1-p} - a(1-p)t)^{-\frac{1}{p-1}}$, this product strictly evaluates to $\mathcal{O}(\varepsilon^{-p})$. Because the latency is $T \propto \varepsilon^{1-p}$, the gradient explosion scales polynomially as $\mathcal{O}(T^{\frac{p}{p-1}})$.

(iii) Combining: either gradients explode exponentially (case i) requiring vanishingly small learning rates, or the trajectory suffers from extreme latency before exploding polynomially (case ii). Both regimes destroy the ability to learn long logic chains. □

Remark 5.2. This dilemma is the formal version of the classical vanishing/exploding gradient problem [2, 8], extended to account for center manifold dynamics. It applies to any shared-weight iterative system, including looped Transformers and latent chain-of-thought architectures.

---

6. Persistence Under Stochastic and Adaptive Optimization

While deep networks lack global strong convexity, their optimization difficulty near a local minimum θ^* is governed by the local strong convexity $\mu = \lambda_{\min}(H(\theta^*))$ and local smoothness $L = \lambda_{\max}(H(\theta^*))$. From the top Lyapunov exponent (Theorem 4.3) and the parameter landscape bounds (Corollary 4.4), the local smoothness behaves as $L \approx \sigma_{\max}^2(J_\theta) = e^{\Theta(D)}$. Concurrently, the local strong convexity for the non-null subspace is safely anchored by the shallow layers to $\mu = \Omega(1)$ (Corollary 4.4). Thus, the local condition number is driven entirely by the exploding smoothness: $\kappa = L/\mu = e^{\Theta(D)}/\Omega(1) = e^{\Theta(D)}$. We apply classical bounds to this local strongly convex basin to assess asymptotic convergence. We also note that the optimal learning rate $\eta = \mathcal{O}(1/L) = \mathcal{O}(e^{-\Theta(D)})$ must be exponentially small, guaranteeing severe numerical underflow on standard finite-precision hardware.

Theorem 6.1 (SGD Persistence). *Locally, SGD with batch size B on a loss with condition number κ converges in $O(\kappa \log(1/\varepsilon) + \sigma^2/(\mu B\varepsilon))$ iterations [3]. Since the local $\kappa = e^{\Theta(D)}$ generically (Corollary 4.4), the deterministic convergence term remains exponential in depth.*

Theorem 6.2 (Momentum Persistence). *Nesterov acceleration reduces the*

local iteration count to $O(\sqrt{\kappa}\log(1/\varepsilon)) = O(e^{\Theta(D)/2}\log(1/\varepsilon))$ [13, Thm 2.2.3]. This is a square-root improvement but remains exponentially intractable.

Theorem 6.3 (Adam Persistence). *Adam [11] normalizes updates coordinate-wise: $\Delta\theta_i = \alpha\hat{m}_i/(\sqrt{\hat{v}_i} + \epsilon)$. This equalization prevents individual coordinate underflow. However:*

(i) Adam's coordinate-wise scaling acts as a diagonal preconditioner $D^{-1}\nabla\mathcal{L}$. This can reduce the local condition number κ by at most $\sqrt{P}$ (where P is parameter dimension), which cannot offset the exponential growth $e^{\Theta(D)}$ [16].

(ii) On severely ill-conditioned landscapes ($\kappa \geq e^{\Theta(D)}$), taking uniform steps of size α guarantees overshooting in high-curvature directions, causing oscillation. Reddi et al. [16] established that Adam can diverge on ill-conditioned problems.

(iii) The AMSGrad variant restores local convergence but requires $O(\sqrt{\kappa}\log(1/\varepsilon))$ iterations [16], matching Nesterov's exponential lower bound.

Corollary 6.4 (Method-Specific First-Order Bound). *The analyzed first-order optimizers (SGD, Nesterov Momentum, AMSGrad) all require at least $\Omega(e^{\Theta(D)/2})$ iterations locally on depth-D logic. The exponential scaling persists across standard adaptive methods.*

---

7. The Optimization Factorization Theorem

7.1 Intermediate Supervision Factorizes Conditioning

Definition 7.1 (Factorized Training). Given $f = g_K \circ \cdots \circ g_1$ with target intermediate values $y_k^* = g_k(\cdots g_1(\mathbf{x}))$, *factorized training* trains each module $\mathcal{N}_\theta^{(k)}$ independently on the sub-problem: minimize $\ell(\mathcal{N}^{(k)}(\mathbf{z}), g_k(\mathbf{z}))$ where inputs $\mathbf{z}$ are the (exact) outputs of the previous stage.

Theorem 7.1 (Optimization Factorization). *Let $f = g_K \circ \cdots \circ g_1$ be a depth-D computation split into K stages, each of depth D/K. Assume $K \mid D$ and the local per-layer condition number satisfies $\kappa_i \leq c$. Then:*

(i) End-to-end training: $\kappa_{\text{end}} = e^{\Theta(D)}$ generically (Corollary 4.4), requiring convergence time $\Omega(e^{\Theta(D)})$.

(ii) Factorized training: each independent sub-problem strictly observes $\kappa_k \leq c^{D/K}$, requiring convergence time $K \cdot \mathcal{O}(c^{D/K}) = \mathcal{O}(K \cdot c^{D/K})$.

(iii) For $K = D$: factorized convergence time $= O(D \cdot c)$ — linear in depth.

(If $K \nmid D$, stages take depths $\lfloor D/K \rfloor$ and $\lceil D/K \rceil$, preserving the identical asymptotic cost $\mathcal{O}(K \cdot c^{\lceil D/K \rceil})$).

Proof.

(i) Direct application of Corollary 4.4.

(ii) Each sub-problem involves a depth-D/K composition. By Theorem 4.1 (sub-multiplicativity), each sub-problem has $\kappa_k \leq c^{D/K}$. Since the K sub-problems are trained independently (no gradient flows between them), the total cost is additive: $\sum_{k=1}^{K} O(\kappa_k \log(1/\varepsilon)) = K \cdot O(c^{D/K} \log(1/\varepsilon))$.

(iii) Setting $K = D$: each sub-problem has depth 1, condition number c, and the total cost is $D \cdot O(c \log(1/\varepsilon))$ — linear in D, independent of exponential scaling. □

Remark 7.2 (Intermediate Supervision Can Be Continuous). The factorization does NOT require non-differentiable checkpoints. One could provide continuous intermediate targets and train with continuous local losses. The key requirement is *intermediate supervision* — access to ground-truth intermediate values — not discreteness. However, in practice, discrete outputs (text tokens, tool results, code outputs) are the natural form of intermediate supervision because:

(a) Discrete tokens are self-verifying (can be checked for correctness).
(b) Discrete tools (calculators, code interpreters) produce exact results.
(c) Chain-of-thought traces provide natural intermediate targets for each reasoning step.

7.2 Agentic Architectures as Optimization Factorization

Theorem 7.3 (Agentic Architecture Advantage). *An agentic system with K tool-use steps, each followed by a neural module of depth $\leq D/K$, achieves:*

(i) Each neural module is trainable in $O(c^{D/K})$ time (within practical reach).

(ii) The discrete tool outputs serve as exact intermediate supervision, eliminating error propagation between stages.

(iii) The total system computes f exactly, with total training cost $O(K \cdot c^{D/K})$ vs. $\Omega(c^D)$ for end-to-end.

Proof. Direct application of Theorem 7.1. The discrete tools ϕ_j compute sub-functions exactly (code interpreter, calculator), providing ground-truth intermediate labels. Each neural module is trained on a shallow sub-problem. □

Remark 7.4 (Exponential Speedup). The factorization provides an exponential optimization speedup. For $D = 20$, $c = 2$, $K = 5$: - End-to-end: $c^D = 2^{20} \approx 10^6$ iterations - Factorized: $K \cdot c^{D/K} = 5 \cdot 2^4 = 80$ iterations

This $> 10,000\times$ speedup is the mathematical explanation for why agentic systems empirically outperform monolithic models on deep reasoning tasks.

---

8. Quantitative Analysis

Table 1. End-to-end vs. factorized training cost (iterations). For illustration, we use an extremely optimistic per-layer condition number $c = 2$. By Edelman's theorem (Theorem 4.3), realistic networks have $c = \Theta(w)$ (e.g., $c \approx 1024$ for width 1024), making the true costs astronomically higher.

Depth D	End-to-end (c^D)	$K = 2$ ($2c^{D/2}$)	$K = 5$ ($5c^{D/5}$)	$K = D$ (Dc)
6	64	16	N/A	12
10	1,024	64	20	20
20	10^6	2,048	80	40
50	10^{15}	6.7×10^7	5,120	100
100	10^{30}	2.3×10^{15}	5.2×10^6	200

Remark 8.1. Modern LLMs train in BF16 with effectively 8–11 bits of mantissa precision. Even assuming $c = 2$, end-to-end training quickly exceeds numerical precision limits beyond $D \approx 20$–50 logical steps. With $K = 5$ factorization (typical agentic tool-use budget), the practical limit extends to $D \approx 100+$.

---

9. Extended Results

We now develop the three main boundary conditions of our framework: why $\kappa_i > 1$ is necessary, how approximate reasoning relaxes the barrier, and whether second-order methods can bypass it.

9.1 Why $\kappa_i > 1$ is Necessary for Nontrivial Logic

Theorem 9.1. *Let $F_i(\mathbf{z}) = \sigma(W_i\mathbf{z}+\mathbf{b}_i)$ be a neural network layer with activation σ. If $\kappa(DF_i(\mathbf{z})) = 1$ for all $\mathbf{z}$ in some open set $U \subset \mathbb{R}^w$, then F_i restricted to U is a conformal affine map (a scalar multiple of an isometry composed with a translation). In particular, F_i cannot implement any nontrivial Boolean sub-function on U.*

Proof. The Jacobian of F_i is $DF_i(\mathbf{z}) = S(\mathbf{z})W_i$, where $S(\mathbf{z}) = \mathrm{diag}(\sigma'(W_i\mathbf{z}+\mathbf{b}_i))$. The condition $\kappa(DF_i) = 1$ means all singular values are equal, so $DF_i(\mathbf{z})^\top DF_i(\mathbf{z}) = \alpha(\mathbf{z})^2 I$ for some scalar $\alpha(\mathbf{z}) > 0$. Equivalently, $DF_i(\mathbf{z})DF_i(\mathbf{z})^\top = \alpha(\mathbf{z})^2 I$.

Let $M = W_i W_i^\top$. The condition $S(\mathbf{z})MS(\mathbf{z}) = \alpha(\mathbf{z})^2 I$ requires the off-diagonals to be zero. Since $\sigma' > 0$ in the active region, we must have $M_{jk} = 0$ for $j \neq k$. Thus, M is diagonal, meaning the rows of W_i are orthogonal.

For the diagonals, $S_{jj}(\mathbf{z})^2 M_{jj} = \alpha(\mathbf{z})^2$, which implies $\sigma'(w_j^\top \mathbf{z}+b_j) = \alpha(\mathbf{z})/\sqrt{M_{jj}}$. Taking the ratio for two non-zero neurons yields $\sigma'(y_j)/\sigma'(y_k) = c$, a constant.

Because w_j and w_k are orthogonal, the pre-activations $y_j = w_j^\top \mathbf{z} + b_j$ and $y_k = w_k^\top \mathbf{z} + b_k$ can vary completely independently. The only way the ratio of two independent variables' functions can be constant is if σ' is constant. If the derivative is constant, the activation function σ must be linear. Thus, no non-linear activation can maintain $\kappa = 1$ over an open set.

For piecewise linear functions like ReLU, σ' is piecewise constant. The condition $\kappa = 1$ forces the active region to have $W_i W_i^\top = \alpha^2 I$ (a scaled isometry). However, as soon as $\mathbf{z}$ crosses into a region where any neuron turns off ($\sigma' = 0$), the matrix loses rank and κ jumps to ∞. Thus, a ReLU layer can be isometric only on a single linear region, reducing it to an affine map.

Therefore, any layer computing a globally nontrivial nonlinear function must have $\kappa(DF_i) > 1$ on a set of positive measure. □

9.2 Approximate Reasoning: Quantitative Relaxation

We quantify how much the optimization barrier weakens when only $(1 - \varepsilon)$-approximate computation is required.

Definition 9.2. A network $\mathcal{N}_\theta$ $(1 - \varepsilon)$-*approximates* a Boolean function f if $\Pr_{\mathbf{x} \sim \mathrm{Uniform}(\{0,1\}^N)}[|\mathcal{N}_\theta(\mathbf{x}) - f(\mathbf{x})| > 1/3] \leq \varepsilon$.

Theorem 9.3 (Approximate Learnability of AC^0). *Let $f \in \mathsf{AC}^0$ (computable by a polynomial-size circuit of constant depth d with unbounded fan-in AND/OR gates). Then f can be $(1 - \varepsilon)$-approximated by a real polynomial of degree $k = O((\log(N/\varepsilon))^d)$ [22]. This polynomial is exactly representable by a depth-2 network with $W = \binom{N}{\leq k}$ neurons. The total optimization cost (gradient descent on the depth-2 network) is quasi-polynomial:*

$$\mathrm{Cost}_{\mathrm{approx}} = N^{O((\log(N/\varepsilon))^d)}$$

compared to $\Omega(c^N)$ for exact end-to-end training of depth-N functions.

Proof. By the Linial–Mansour–Nisan theorem [22], every $f \in \mathsf{AC}^0$ has Fourier spectrum concentrated on coefficients of degree $\leq k = O((\log(N/\varepsilon))^d)$: specifically, $\sum_{|S|>k} \hat{f}(S)^2 \leq \varepsilon^2$. The truncation $\tilde{f}(\mathbf{x}) = \sum_{|S|\leq k} \hat{f}(S)\chi_S(\mathbf{x})$ is a real polynomial of degree k satisfying $\mathbb{E}[(f - \tilde{f})^2] \leq \varepsilon^2$. By Markov's inequality, $\Pr[|f - \tilde{f}| > 1/3] \leq 9\varepsilon^2$.

A degree-k polynomial on $\{0,1\}^N$ can be exactly computed by a single-hidden-layer (depth-2) network with $W = \binom{N}{\leq k}$ neurons (one per monomial). The depth is constant, so the depth-based condition number is $\kappa \leq c^2$ — the barrier from Theorem 4.1 does not apply. Instead, the optimization cost is dominated by the *parameter count* $P = O(NW) = O(N\binom{N}{\leq k})$. For $k = O((\log(N/\varepsilon))^d)$ and constant d, W is quasi-polynomial in N: $W = N^{O((\log(N/\varepsilon))^d)}$. The total optimization cost (including sample complexity and gradient computation) is

therefore quasi-polynomial, compared to the $\Omega(c^N)$ cost for exact end-to-end training of depth-N functions.

More precisely, the condition number of the depth-2 wide network is controlled by the Gram matrix of the monomial features, which has condition number $\kappa_W = O(W^2)$ in the worst case. The total optimization cost is $O(c^2 \cdot W^2 \cdot \log(1/\varepsilon)) = N^{O((\log(N/\varepsilon))^d)}$ — quasi-polynomial. □

Remark 9.4. For functions *outside* AC^0, the quasi-polynomial bound fails. For example, parity is the full Fourier character $\chi_{[N]}$, so its only non-zero Fourier coefficient lies at exactly degree N. By elementary Fourier analysis, any constant-error approximation of parity under the uniform distribution rigorously requires degree $\Omega(N)$. In these cases, the full exponential barrier applies even for approximate computation. This provides a precise complexity-theoretic boundary: AC^0 functions are approximately learnable by gradient descent; functions outside AC^0 are not.

9.3 Second-Order Methods

Theorem 9.5 (Newton's Method — Condition Number Independence). *Let $\mathcal{L} : \mathbb{R}^P \to \mathbb{R}$ be twice continuously differentiable with β-Lipschitz Hessian ($\|\nabla^2\mathcal{L}(\theta) - \nabla^2\mathcal{L}(\theta')\| \leq \beta\|\theta - \theta'\|$). Newton's method $\theta_{t+1} = \theta_t - [\nabla^2\mathcal{L}(\theta_t)]^{-1}\nabla\mathcal{L}(\theta_t)$ achieves quadratic convergence within the Kantorovich radius [13]:*

$$\|\theta_{t+1} - \theta^*\| \leq \frac{\beta}{2\mu}\|\theta_t - \theta^*\|^2$$

where $\mu = \lambda_{\min}(\nabla^2\mathcal{L}(\theta^*))$. The iteration count to reach ε-accuracy from within the convergence basin is:

$$T_{\text{Newton}} = O\left(\log\log\frac{1}{\varepsilon}\right)$$

which is independent of the condition number κ.

Theorem 9.6 (Newton's Method — Practical Cost). *Each Newton iteration requires:*

(i) Computing $\nabla^2\mathcal{L}(\theta) \in \mathbb{R}^{P\times P}$: cost $O(P^2)$ via Hessian-vector products or $O(P^2 \cdot |\mathcal{D}|)$ explicitly.

(ii) Solving the linear system $\nabla^2\mathcal{L} \cdot \Delta\theta = -\nabla\mathcal{L}$: cost $O(P^3)$ via direct factorization, or $O(P^2 \cdot \sqrt{\kappa_H})$ via conjugate gradient, where κ_H is the Hessian condition number.

(iii) Total cost: $T_{\text{Newton}} \cdot O(P^3) = O(P^3 \log\log(1/\varepsilon))$.

Corollary 9.7 (Crossover Point). *Newton's method is computationally preferable to first-order methods when the condition number exceeds the squared parameter count:*

$$\kappa > P^2$$

Since $\kappa = e^{\Theta(D)}$ generically and $P = O(Dw^2)$ for width-w networks, the crossover occurs at:

$$D_{\text{crossover}} = \Theta(\log P) = \Theta(\log(Dw^2)) \approx \Theta(\log w)$$

For modern architectures ($w \sim 10^3$–10^4, $D \sim 10$–100), $P \sim 10^7$–10^9, giving $D_{\text{crossover}} \approx 14$–$20$.

Theorem 9.8 (Newton's Method Does Not Fully Bypass the Barrier). *Despite condition-number-independent iteration count, Newton's method faces three obstructions for deep logic training:*

(i) (Convergence basin). The standard Kantorovich theorem requires the initial iterate to satisfy $h = \frac{\beta\|H^{-1}\|^2\|\nabla\mathcal{L}\|}{2} \leq \frac{1}{2}$ for quadratic convergence. For deep networks, the Hessian Lipschitz constant β explodes with depth as $\Omega(\lambda_{\max}^{3D})$ due to triple products of Jacobians along exploding directions. Although the inverse Hessian norm in the non-null subspace is anchored at $\|H^{-1}\| = 1/\mu = \mathcal{O}(1)$ by the final layer (Corollary 4.4), the Kantorovich factor $h \propto \beta\|H^{-1}\|^2\|\nabla\mathcal{L}\| \propto \lambda_{\max}^{3D} \cdot \mathcal{O}(1) \cdot \|\nabla\mathcal{L}\|$ still explodes exponentially in D. The condition $h \leq \frac{1}{2}$ can only be met if the initial gradient $\|\nabla\mathcal{L}\|$ is exponentially small, and the convergence radius $r = 2\mu/\beta = \mathcal{O}(1)/\Omega(\lambda_{\max}^{3D}) = e^{-\Theta(D)}$ is exponentially narrow—meaning Newton's method strictly requires an exponentially precise initial guess, which is overwhelmingly improbable from random initialization.

(ii) (Saddle points). For non-convex losses, Newton's method converges to any critical point (minima, maxima, or saddle points). In high-dimensional random landscapes, the number of saddle points grows exponentially with the parameter count P [26], meaning saddle points dominate the landscape. Cubic regularization (Nesterov–Polyak [13]) avoids strict saddle points but adds $O(\kappa^{1/3})$ overhead, partially reintroducing condition number dependence.

(iii) (Per-iteration cost). For $P \sim 10^9$ (typical LLM), each Newton step costs $O(P^3) \sim 10^{27}$ FLOPs — approximately 10^{15} GPU-years at current hardware. Even rank-k approximate methods (L-BFGS, K-FAC) reduce per-step cost to $O(kP)$ but can only reduce the effective condition number by factor k, not exponentially.

Therefore, while Newton's method eliminates condition number dependence from the iteration count, it reintroduces exponential cost through the convergence basin size and per-iteration complexity. The net effect: second-order methods shift the exponential cost from iterations to per-step computation and initialization quality, without eliminating it. □

---

10. Conclusion

This paper establishes that the gap between neural network *representability* and *learnability* is fundamentally optimization-theoretic, not representational.

What we proved:

1. **No representational barrier** (Theorem 3.1). The multilinear extension of any Boolean function — regardless of logical depth — has Lipschitz constant $L \leq \sqrt{N}$. Smooth, well-conditioned continuous solutions always exist. The Universal Approximation Theorem is not the bottleneck.

2. **Generic exponential conditioning** (Theorems 4.1, 4.3). For feedforward networks with standard initialization, the end-to-end condition number satisfies $\kappa = e^{\Theta(D)}$ with high probability, by the Furstenberg–Kesten theorem and the positivity of the Lyapunov exponent gap. This is a generic (probabilistic) property, not an unconditional lower bound.

3. **The recurrent dilemma** (Theorem 5.1). Shared-weight iterative architectures maintaining discrete logical states face either gradient explosion ($\lambda_{\max} > 1$, exponential in T) or polynomial gradient explosion ($\lambda_{\max} = 1$, polynomial center-manifold escape). Both render training intractable for large sequence lengths.

4. **First-order optimizer persistence** (Theorems 6.1–6.3). SGD, Nesterov momentum, and Adam each gain at most constant-factor improvements (momentum and Adam: $\sqrt{\kappa}$). The exponential scaling $e^{\Theta(D)}$ persists across all first-order methods.

5. **Optimization factorization** (Theorem 7.1). Intermediate supervision — whether via discrete tool outputs or continuous intermediate targets — decomposes one depth-D problem with cost c^D into K independent sub-problems with total cost $K \cdot c^{D/K}$. For $K = D$, the cost is linear in D.

6. **Necessity of $\kappa > 1$** (Theorem 9.1). Any layer computing a nontrivial nonlinear function must have per-layer condition number $\kappa_i > 1$ on a set of positive measure. Perfectly isometric layers are restricted to conformal affine maps.

7. **Complexity-theoretic boundary** (Theorem 9.3). AC^0 functions are approximately learnable in quasi-polynomial time via Fourier truncation. Functions outside AC^0 (parity, modular arithmetic) face the full exponential barrier even for approximate computation.

8. **Second-order methods shift, but do not eliminate, the barrier** (Theorem 9.8). Newton's method removes κ-dependence from the iteration count but reintroduces exponential cost through the shrinking Kantorovich convergence basin ($r = e^{-\Theta(D)}$) and $O(P^3)$ per-iteration complexity.

The practical implication. The industry pivot from monolithic LLM scaling to agentic architectures — tool-use, chain-of-thought, multi-agent orchestration — is not merely an engineering heuristic. It is the natural instantiation of optimization factorization (Theorem 7.1): each tool call or reasoning checkpoint provides intermediate supervision that breaks an intractable deep optimization into

tractable shallow sub-problems. Not because continuous learning is *impossible*, but because it is *exponentially slower* without intermediate supervision.

---

11. Open Questions

Open Question 1 (Tight Lyapunov Gap). Theorem 4.3 establishes $\log \kappa = \gamma D + o(D)$ where $\gamma = \lambda_1 - \lambda_w > 0$. What is the precise value of γ for standard initializations (Xavier, He, LeCun) as a function of the width w? Glorot and Bengio [7] provide empirical estimates; a closed-form expression for $\gamma(w)$ would yield sharp critical depth predictions.

Open Question 2 (Condition Number During Training). Our analysis establishes $\kappa = e^{\Theta(D)}$ at initialization. Does this exponential conditioning persist throughout training, or can gradient descent implicitly regularize toward lower-κ solutions? If the loss landscape has a "progressive sharpening" phenomenon [27], κ may increase during training, strengthening our results. Conversely, if implicit regularization reduces κ, the effective barrier may be weaker than our generic bound suggests.

Open Question 3 (Beyond AC^0). Theorem 9.3 shows that AC^0 functions are quasi-polynomially learnable via Fourier truncation. What is the precise complexity-theoretic boundary for gradient-based learnability? Is the class of efficiently learnable functions exactly AC^0, or does it extend to TC^0 (threshold circuits) or NC^1 (logarithmic-depth circuits)? The answer depends on whether majority-type functions, which are in TC^0 but not AC^0, admit low-degree polynomial approximations under non-uniform distributions.

Open Question 4 (Optimal Factorization Granularity). Theorem 7.1 shows that K intermediate checkpoints reduce cost from c^D to $K \cdot c^{D/K}$, minimized at $K = D/\ln c$ giving cost $eD/\ln c$. In practice, each checkpoint incurs overhead (tool-call latency, context switching). What is the optimal K^* that minimizes total wall-clock time $K \cdot c^{D/K} + K \cdot t_{\text{overhead}}$? For $t_{\text{overhead}} \gg 1$, the optimal K^* is sublinear in D, suggesting that a small number of high-quality tool calls outperforms many shallow ones.

Open Question 5 (Continuous Intermediate Supervision). Remark 7.2 notes that intermediate supervision need not be discrete. Can continuous intermediate targets (e.g., soft labels, feature distillation) achieve the same factorization benefit as discrete tool outputs? If so, this would suggest that the optimization advantage of agentic architectures can be replicated within a purely differentiable framework, provided intermediate ground-truth signals are available.

Open Question 6 (Quantum and Analog Computation). Our results assume digital floating-point arithmetic. Quantum computing operates with continuous amplitudes and exponentially large Hilbert spaces. Does the condition number barrier persist for quantum gradient descent (variational quantum

eigensolvers), or does the exponential state space provide a natural bypass? The "barren plateau" phenomenon in variational quantum circuits [23] suggests that an analogous barrier exists, but the precise relationship to our $\kappa = e^{\Theta(D)}$ result is unknown.

Open Question 7 (Empirical Validation). Our theoretical predictions — specifically, that end-to-end LLM reasoning fails at depth $D \approx 15$–30 sequential logical steps while agentic systems with $K = 5$ checkpoints extend to $D \approx 100+$ — should be empirically testable. Designing controlled experiments on synthetic logic tasks (iterated arithmetic, multi-step planning, nested conditionals) with varying depth D and number of tool-use steps K would provide direct empirical validation or refutation of the optimization factorization theorem.

References

[1] Balduzzi, D. et al. "The Shattered Gradients Problem." *Proc. ICML*, 2017.

[2] Bengio, Y., Simard, P., Frasconi, P. "Learning Long-Term Dependencies with Gradient Descent is Difficult." *IEEE Trans. Neural Networks*, 1994.

[3] Bottou, L., Curtis, F. E., Nocedal, J. "Optimization Methods for Large-Scale ML." *SIAM Review*, 2018.

[4] Choromanska, A. et al. "The Loss Surfaces of Multilayer Networks." *AISTATS*, 2015.

[5] Cybenko, G. "Approximation by Superpositions of a Sigmoidal Function." *MCSS*, 1989.

[6] Eldan, R., Shamir, O. "The Power of Depth for Feedforward Neural Networks." *COLT*, 2016.

[7] Glorot, X., Bengio, Y. "Understanding the Difficulty of Training Deep Networks." *AISTATS*, 2010.

[8] Hochreiter, S. "Untersuchungen zu dynamischen neuronalen Netzen." TU Munich, 1991.

[9] Hornik, K. et al. "Multilayer Feedforward Networks are Universal Approximators." *Neural Networks*, 1989.

[10] Hirsch, M. W., Smale, S., Devaney, R. L. *Dynamical Systems and Chaos.* Academic Press, 2012.

[11] Kingma, D. P., Ba, J. "Adam." *ICLR*, 2015.

[12] Edelman, A. "Eigenvalues and Condition Numbers of Random Matrices." *SIAM Journal on Matrix Analysis and Applications*, 9(4):543–560, 1988.

[13] Nesterov, Y. *Introductory Lectures on Convex Optimization.* Springer, 2004.

[14] Palis, J., de Melo, W. *Geometric Theory of Dynamical Systems.* Springer, 1982.

[15] Pascanu, R., Mikolov, T., Bengio, Y. "On the Difficulty of Training RNNs." *ICML*, 2013.

[16] Reddi, S. J. et al. "On the Convergence of Adam and Beyond." *ICLR*, 2018.

[17] Telgarsky, M. "Benefits of Depth in Neural Networks." *COLT*, 2016.

[18] Furstenberg, H. and Kesten, H. "Products of Random Matrices." *Ann. Math. Statist.*, 31(2):457–469, 1960.

[19] Oseledets, V. I. "A Multiplicative Ergodic Theorem. Lyapunov Characteristic Numbers for Dynamical Systems." *Trans. Moscow Math. Soc.*, 19:197–231, 1968.

[20] Furstenberg, H. "Non-Commuting Random Products." *Trans. AMS*, 108(3):377–428, 1963.

[21] Le Page, É. "Théorèmes limites pour les produits de matrices aléatoires." *Springer Lecture Notes in Mathematics*, 928:59–84, 1982.

[22] Linial, N., Mansour, Y., and Nisan, N. "Constant Depth Circuits, Fourier Transform, and Learnability." *J. ACM*, 40(3):607–620, 1993.

[23] McClean, J. R. et al. "Barren Plateaus in Quantum Neural Network Training Landscapes." *Nature Communications*, 9:4812, 2018.

[24] Schoenholz, S. S. et al. "Deep Information Propagation." *ICLR*, 2017.

[25] Poole, B. et al. "Exponential Expressivity in Deep Neural Networks through Transient Chaos." *NeurIPS*, 2016.

[26] Dauphin, Y. N. et al. "Identifying and attacking the saddle point problem in high-dimensional non-convex optimization." *NeurIPS*, 2014.

[27] Cohen, J. et al. "Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability." *ICLR*, 2021.